Proactive Agent: Learning to Proactively Predict Implicit Human Intents

Anonymous ARR submission

Abstract

Agents powered by large language models have shown remarkable capabilities in solving complex tasks. However, most agent systems remain reactive and are unable to proactively provide assistance to humans, limiting their effectiveness in scenarios requiring foresight and autonomous decision-making. This paper proposes a novel formalization for developing proactive agents using a data-driven approach. Specifically, we first develop a data collection and generation pipeline to obtain human activity events and their associated intents, resulting in the creation of a dataset ProactiveBench with $11,876$ events. To test various models' performance in predicting human intent, we build an automatic evaluator with annotated data, which shows a 76% agreement rate with human annotators in our experiments. We fine-tune the LLaMA-3-8B model on the ProactiveBench to obtain LLaMA-3-Proactive, which can predict human needs based on the given activity traces. It achieves a 70.34% pass rate on the test set of ProactiveBench, demonstrating the effectiveness of our method. This work aims to advance autonomous agents' ability to understand and anticipate human needs, leading to more intuitive and efficient interactions. The code and dataset will be released after the blind review process.

1 Introduction

The emergence of large language models (LLMs) like ChatGPT (OpenAI, 2022) has significantly advanced the development of autonomous agents. These agents can understand human instructions, make plans, explore environments, and utilize tools to solve complex tasks (BabyAGI, 2023; Wu et al., 2023; Li et al., 2023; AutoGPT, 2023; XAgent-Team, 2023), showing substantial potential in various applications such as robotics (Firoozi et al., 2023), personal assistants (Li et al., 2024), and process automation (Ye et al., 2023b).

Current agent systems are predominantly reactive, responding to user inputs and functioning in a manner where their actions are triggered by specific queries or commands (Ouyang et al., 2022), as illustrated in Figure 1 left. This interaction style is inherently user-driven, focusing on addressing users' immediate needs and providing instant reactions. However, reactive agents lack proactivity and are unable to anticipate future needs or actions, limiting their effectiveness in scenarios that require foresight or autonomous decision-making.

In this work, we propose to develop the proactive agent, which can anticipate user requirements and take the initiative, suggesting actions or providing information without explicit requests, thereby providing a more seamless user experience, as illustrated in Figure 1 right. Specifically, we introduce a novel formalization for developing proactive agents from a data-driven perspective. Our approach centers on constructing a Proactive Supervised Fine-Tuning (SFT) dataset, ProactiveBench, tailored to address the main challenges in building proactive systems. Firstly, the proactive agent should precisely predict human intents, as false predictions will increase the cognitive burden for human interest. Secondly, the essence of proactivity lies in balance. It is crucial that agents do not become overly aggressive in their assistance, which could lead to unnecessary interruptions. Lastly, the vast variability in human activities across different environments demands a dataset that encompasses a wide range of scenarios, ensuring that our agents can adapt to diverse contexts.

To address these challenges, we have developed a data synthesis pipeline that meticulously captures human activities in three distinct settings: coding, writing, and smart home. This pipeline generates human activity events, simulating real-world scenarios and producing ProactiveBench, which consists of $11,876$ entries. Each entry contains the predicted human intents and the event trace

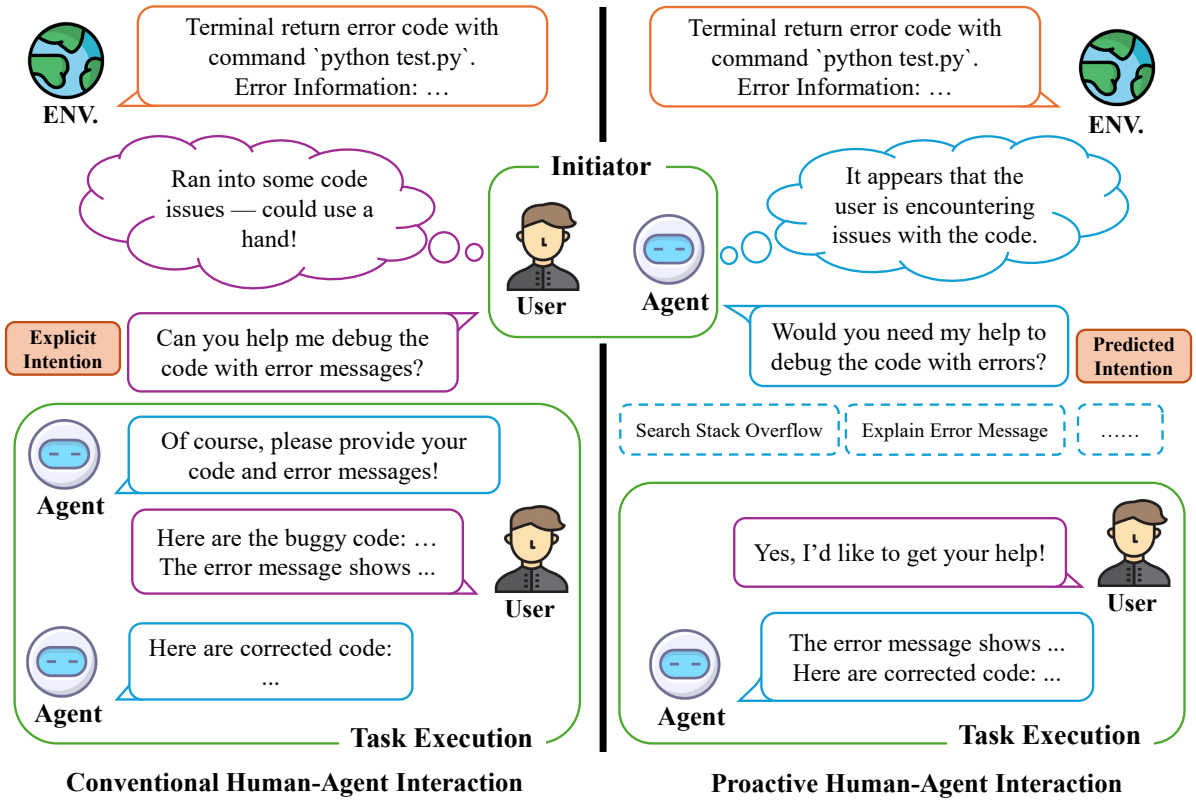


Figure 1: Comparison of agent systems with two types of human-agent interaction. Reactive agents passively receive user queries and then generate responses. Proactive agents infer the user's implicit intents based on environmental information and propose possible assistance requests accordingly.

that happened in the environment. We fine-tuned the LLaMA-3-8B (Meta, 2024) model on ProactiveBench to obtain LLaMA-3-Proactive, aiming to predict human intents with a nuanced understanding of different environments, thereby enhancing the model's robustness and flexibility.

To validate the performance of our model, we first constructed an automatic evaluator that simulates human judgment on the predicted intents. This evaluator was trained and validated on an additional set of $1,400$ manually annotated data entries, where human annotators provided judgments on whether the predicted intents were acceptable. The evaluator achieved a 76% agreement rate with human judgments annotated by human annotators, demonstrating its reliability in assessing the quality of predictions. Based on the automatic evaluator, we compare the performance of our model with other existing LLMs on the ProactiveBench. The experimental results show that our model achieved the highest pass rate of 70.34% compared with all existing open-source and close-source LLMs, underscoring the effectiveness of our data-driven approach in developing proactive agents. This success not only validates our method but also highlights the potential of proactive agents in enhancing user experiences across various applications.

2 Related Works

Recent advancement in large language models (LLMs) (OpenAI et al., 2023; Chowdhery et al., 2022; Touvron et al., 2023; Zeng et al., 2022) has shown great progress in complex reasoning, task planning (Wei et al., 2023; Gao et al., 2023; Yao et al., 2022, 2023; Liu et al., 2023; Ye et al., 2023a,b), tool utilization (Schick et al., 2023; Qin et al., 2023b,c; Qian et al., 2023b), etc. Consequently, a growing number of agent systems have been developed to tackle diverse tasks like automatic web search (Qin et al., 2023a), software development (Qian et al., 2023a; Chen et al., 2023), behavior simulation (Park et al., 2023). Despite these advancements, current agents work in a reactive manner (i.e., passively accept human instructions) and often struggle to proactively grasp user intentions from environment observations. This

issue is particularly pronounced in scenarios that require intensive human-agent interactions, such as coding, writing, and smart home. In response to this challenge, in this work, we propose a novel dataset named ProactiveBench that predicts human intentions directly from the observed environment (e.g., human activity events). Different from several existing works (Qian et al., 2024; Wu et al., 2021; Chen et al., 2020; Wiethof et al., 2021) which enable multi-turn interactions to better understand the ambiguous instructions, we focus on anticipating the user intentions based on the observed environment. Working in the proactive manner, the agent systems can reduce the user's cognitive burden of providing explicit instructions and promote more effective assistance.

3 Task Definition

As illustrated in Figure 1, conventional LLM-based agent systems typically derive their tasks directly from a human intention I, necessitating explicit expression of user requirements. In contrast to the conventional approach, we explore an alternative scenario where the agent autonomously infers user requirements, and names it as the proactive agent.

The task of the proactive agent is to predict potential human intention I based on the current environment event e_T and a sequence of historical environment events $E = \{e_0, e_1, ..., e_{T-1}\}$. These events encompass a wide range of data, including user actions and queries, essential information regarding the status of the environment, and any historical interactions between the user and the agents. The task objective then becomes predicting the potential human intention given the current environment event e_T and the historical event trace E. In practice, proposing unnecessary user intents can impose additional cognitive burdens on the user. Hence, we not only aim to predict what the user's intent might be but also to first determine whether an intention exists.

4 Dataset Construction

Our dataset ProactiveBench is constructed to enhance the proactive capability of LLMs. It consists of three typical proactive domains: coding, writing, and smart home. The construction of ProactiveBench is based on the three crucial components:

(1) **Environment Gym**: It provides a gym for proactive agents to interact. It has two functionalities: (i) event generation: generating potential environment event sequences tailored to specific scenarios; (ii) task execution: executing tasks proposed by the proactive agents to generate subsequent events.

(2) **Intent Prediction Agent**: It, based on the historical event trace and the current environment events, aims to generate potential user intents, i.e., tasks that the user might want the agent to proactively assist with.

(3) **User Agent**: It emulates users based on the user's characteristics to decide whether to execute the tasks proposed by the agent.

The dataset construction procedure works in a interactive manner. The environment gym generates a sequence of events and the intent prediction agent predicts tasks the user might propose. Then, the user agent decides whether to accept the predicted tasks, and if a task is accepted, the environment gym executes the task and generates the subsequent events. Through multiple iterations of this interactive process, we could derive the proactive interaction data automatically. Finally, as shown in Table 1, ProactiveBench comprises a total of 653 data instances, with an average of 18.15 events per instance, amounting to $11,876$ events in total. In the following sections, we introduce the details of each component.

4.1 Environment Gym

4.1.1 Event Generation

The event generation contains two distinct phases: (1) generating specific scenarios to establish the fundamental environment based on the given overall user goal, and (2) generating detailed events according to the established scenarios, tailored to the interactions with the intent prediction agent and the user agent.

Scenario Generation Scenario generation aims to outline the events that should unfold throughout the interaction process. For coding and writing scenarios, we simply collect 20 daily tasks from humans as seed data. Then we prompt GPT-4o(OpenAI, 2024) to create a variety of daily tasks and work environments that simulate typical user interactions with computer systems. These scenarios are straightforward and focus primarily on interactions within digital environments, thus providing a clear framework for expected user actions. For the smart home setting, however, we delve into a more complex simulation. Our objective here is to emulate a smart home environment rich with a

Setting	Inst. Num (Train/Test)	Total Events Num	Avg. Tokens Num	Intent Density
Coding	163 / 16	3,580	25,763	0.487
Writing	184 / 10	3,768	23,813	0.299
Smart Home	272 / 8	4,528	31,212	0.157
Total	619 / 34	11,876	27,566	0.354

Table 1: Comprehensive statistics of our datasets in three distinct settings: Coding, Writing, and Smart Home, summarized across key metrics. The datasets collectively comprise 619 training instances and 34 test instances, with a total of $11,876$ events. Intent density is calculated as the ratio of turns with user intents to the total turns in the test set. The average number of tokens takes both input and output during the generation into consideration, and the tokens are quantified using the OpenAI $o200k_base$ tokenizer.

variety of furniture and appliances. We leverage a collection of high-quality home automation data sourced from the Home Assistant community's public repository[1]. The blueprints detail the functionalities and interplay of various smart home devices, from the lighting systems to the water heaters and beyond. We randomly sample the blueprints and prompt GPT-4o to construct textual scenarios that seamlessly integrate these devices, simulating a realistic smart home experience.

Detailed Events Generation The creation of detailed events is meticulously aligned with the scenario descriptions and is continuously refined through iterative interactions with the user agent and intent prediction agent. As depicted in Figure 2, the Gym's generation of new events is informed by a combination of factors. Specifically, our language model-driven Gym takes scenario descriptions, the sequence of historical events, the current status of the environment, and the interactions with other components together, to generate logically correct and fluent events. With every new task carried out by the user agent and intent prediction agent, the Gym updates its internal representation of the environment's status. The Gym utilizes the updated status, the ongoing interactions with the agent, and the context provided by scenario descriptions to generate the next event, denoted as e_t, more consistently. This comprehensive approach ensures that each subsequent event is not only appropriate but also contributes to a coherent and logical progression within the scenario.

In our event generation framework, ensuring that the generated events are both reasonable and accurately reflect the characteristics of the predefined scenarios is crucial. To achieve this, we have taken several measures to ensure the rationality and consistency of the event space, particularly by aligning it with actual data to validate our simulated environments.

For the coding and writing environments, our approach involves aligning the events generated by the simulation directly with data collected through Activity Watcher[2]. Activity Watcher allows us to capture the details of user interactions with computer systems, including keyboard and mouse operations, visited web pages, and used development tools. By comparing these actual collected data, we can adjust and refine our event generation algorithms by prompting and providing samples of the real collected data to the language model, ensuring that the generated events realistically reflect user behavior patterns in these environments.

For smart home scenarios, our approach is slightly different. Given the complexity of directly collecting interaction data for smart home devices in the real world, we instead utilize blueprints publicly available from the Home Assistant community. These blueprints provide detailed descriptions of smart home device usage scenarios, including the logical relationships between devices and user interaction patterns. By parsing these descriptions from the blueprints, we construct an event space that reflects the various events that can occur in a smart home environment. We then align this event space with our simulation environment with prompting to ensure that the generated events are not only logically reasonable but also accurately simulate the actual use of smart home devices.

4.1.2 Task Execution

Another key functionality of the environment gym is to simulate task execution. This process is initiated when the user agent accepts a prediction from the intent prediction agent, which will start multi-

[1] https://community.home-assistant.io/c/blueprints-exchange/53

[2] https://github.com/ActivityWatch/activitywatch

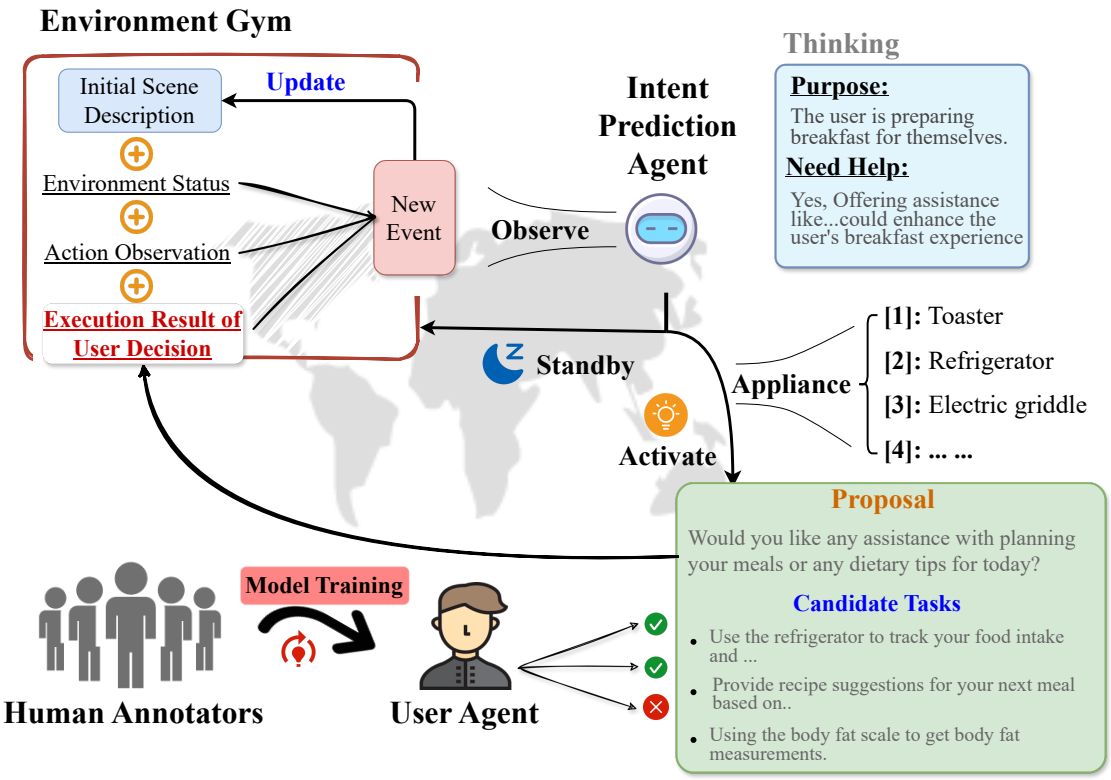


Figure 2: Overview of the data generation process. Taking the smart home scenario as an example, this process includes modules such as the initial scene setup, events generation, agent observation, appliance invocation, and predicting intents with candidate tasks.

ple interactions between the agent and the gym.

Specifically, once the task execution phase commences, the intent prediction agent will generate actions using the tools provided by the gym. For coding and writing tasks, these tools include a set of built-in utilities such as file system tools and command-line tools. For home automation tasks, the tools are a set of interfaces for smart appliances, retrieved based on the task at hand.

The gym then simulates the completion of these actions by considering historical events, the current state of the environment, and other relevant factors, subsequently generating new events about the execution result. The intent prediction agent, upon receiving these events, continues to execute further actions until the corresponding intent is fulfilled.

4.2 Intent Prediction Agent

The second component in our data generation pipeline is the agent that can predict user intents according to a series of events, which developed through direct prompting of GPT-4o. As detailed in Figure 3, upon the agent receiving new event e_t at turn t with the event listener, it first updates its memory with the event. To better predict user intents, our agent also summarizes and stores the user's characteristics and preferences in the memory. Combining new events with historical ones and conversations with the user, the agent incorporates user characteristics to determine possible user intents. If so, the agent will proactively formulate potential intents based on the memory and user characteristics. The predicted intent will be presented to the user and awaits for user's response about whether to accept one of the predicted intents. If one of the predictions is accepted, the agent will execute a task according to the intent within the Environment Gym and update a new event about the task. During the data generation, the agent would constantly receive events from the Gym and propose predictions about the user intents.

4.3 User Agent

The user agent is designed to emulate the responses of actual users by selecting from the predicted intents, as illustrated in Figure 2. Upon receiving

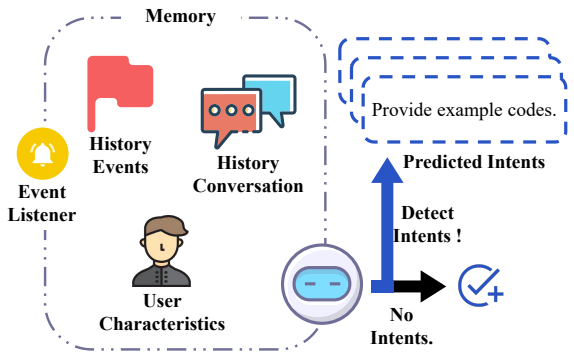


Figure 3: Overview of the framework for Intent Prediction Agent. The agent listens to the new events and updates its memory to predict user intents.

these predicted intents, the critical role of the User Agent is to determine whether to accept or reject these suggestions. This decision-making process is critical, as it not only dictates the subsequent sequence of events in the dataset construction but also serves as a metric for evaluating the accuracy of proactive agent systems. If the User Agent accepts the intents (fully or partially), the proactive agent then executes the accepted intents within the Environment Gym. If, however, the User Agent declines the suggested assistance, the Environment Gym proceeds to generate new events autonomously, without any interventions. This structured methodology ensures a tightly integrated simulation process, crucial for the creation of realistic and coherent scenarios, where the dynamics between user simulation and event generation play a pivotal role.

To facilitate automatic data generation, we prompt the model to produce simulated user agent responses. Further details regarding the assessment of the user agent are available in Section 5.2.

5 Experiments

5.1 Proactive Agent Evaluation

Metrics. To evaluate the quality of prediction, we present four metrics, which are defined as follows:

- **Coverage Ratio** This represents the proportion of rounds where the agent proposes candidate tasks to the total rounds where human intents are present. It reflects the agent's proactivity in recognizing and responding to human intents.
- **False-alarm Ratio** This is the ratio of rounds where the agent proposes candidate tasks in the absence of human intents to the total rounds where the agent proposes candidate tasks. It measures the frequency at which the agent erroneously proposes tasks when there are no explicit human intents.
- **Accuracy** This is the ratio of the number of candidate tasks proposed by the agent that the human accepts to the total number of candidate tasks proposed by the agent. It indicates the accuracy and relevance of the tasks proposed by the agent.
- **Pass Rate** This is the ratio of rounds where at least one of the candidate tasks proposed by the agent is accepted to the total rounds where human intents are present. It represents the frequency at which the agent's proposed tasks are accepted by humans when there are explicit human intents.

Setting. We use the training set of ProactiveBench to train our agent model based on the LLaMA-3-8B-Instruct model, the detailed training parameters can be found in Appendix Appendix B. The automatic evaluation of these four metrics relies on the simulated user agent judgment. All models are evaluated in our test split of the dataset, which contains 655 entries. We employ the same prompt template across all models. The temperature is set to 0.1 during testing.

Result. Table 2 presents a comparison of various models across four metrics. Our model distinguishes itself by achieving an unparalleled Coverage Ratio of 82.63%, indicating its exceptional proficiency in recognizing a wide array of user intents. This capability ensures that nearly all user queries receive some form of response, highlighting the model's extensive understanding and responsiveness to user needs. Additionally, our model leads in Pass Rate with a remarkable 70.34%, suggesting that it not only identifies user intents but also offers responses that are most likely to be deemed acceptable or useful by users. This balance underscores the model's effectiveness in delivering relevant and contextually appropriate assistance.

However, the pursuit of broad coverage and high pass rates comes with its trade-offs. Our model exhibits a False-alarm Ratio of 42.97%, which, while not the highest, indicates a propensity to occasionally identify intents where there might be none, leading to unnecessary or irrelevant responses. Furthermore, its Accuracy stands at 30.74%, reflecting the challenges in maintaining precision while aiming for extensive coverage and high acceptance rates.

Model Name	Coverage Ratio (%↑)	False-alarm Ratio (%↓)	Acc. (%↑)	Pass Rate (%↑)
GPT-3.5-Turbo(Brown et al., 2020)	26.69	18.88	**62.33**	23.73
GPT-4o (OpenAI, 2024)	41.95	20.49	54.29	36.44
Claude-3-Sonnet (Anthropic, 2024)	22.60	**10.00**	51.87	19.23
DeepSeek-v2 (DeepSeek-AI, 2024)	53.39	20.92	51.89	44.92
LLaMA-3-8B (Touvron et al., 2023)	44.07	30.23	37.67	32.63
LLaMA-3-70B(Touvron et al., 2023)	76.39	15.50	54.00	67.59
Qwen2 72B (Bai et al., 2023)	62.29	32.33	41.92	50.00
Mixtral 8x22B (Jiang et al., 2024)	57.20	27.53	49.16	44.92
LLaMA-3-Proactive	**82.63**	26.68	53.82	**70.34**

Table 2: Evaluation results of different models on four key metrics: Coverage Ratio, False-alarm Ratio, Accuracy, and Pass Rate. Our model significantly outperforms other models in terms of Coverage Ratio (82.63%) and Pass Rate (70.34%), demonstrating its superior ability to recognize and respond to human intents with a high acceptance rate.

Model	Agreement Ratio (↑)
GPT-3.5-Turbo	0.40
GPT-4o	0.38
Claude-3-Sonnet	0.35
LLaMA-3-70B	0.53
Mixtral 8x22B	0.42
Qwen2 72B	0.30
Deepseek-v2	0.33
Ours	**0.76**

Table 3: Evaluation results of different models on our test set for User Agent Assessment.

In contrast, other models like GPT-3.5-Turbo and LLaMA-3-8B demonstrate strengths in specific areas. GPT-3.5-Turbo, for instance, boasts the highest Accuracy at 39.73%, suggesting it is more precise in its responses, albeit with a lower Coverage Ratio and Pass Rate. DeepSeek-v2 and Qwen2 show a tendency towards higher Coverage Ratios, with DeepSeek-v2 at 53.39% and Qwen2 leading at 62.29%, yet both struggle with higher False-alarm Ratios and lower Accuracy, demonstrating the inherent challenge in optimizing across all four metrics simultaneously.

A more detailed, categorized result is shown in Table 4 in Appendix. It can be read from the table that all the models perform well with scenes like coding and writing. But surprisingly, models except Qwen2 seem cannot perform well in smart home scenes, which might result from being short of related knowledge. It is worth noting that Claude-3-Sonnet could not raise a single proposal during the evaluation of the smart home scenario, leading to the lowest false-alarm ratio in the overall metric.

5.2 User Agent Training and Assessment

To automatically evaluate whether the generated user intents and their timing are appropriate, we seek to train a model that is capable of accurately imitating user judgments. To this end, we apply the user-annotated data to train Llama-3-Instruct 8B(Touvron et al., 2023) and compare it with several baselines to show its superiority.

User Agent Data Annotation To ensure the user agent aligns closely with human judgment, we generate and annotate a dedicated dataset to indicate whether humans would accept the predicted intents. Specifically, each prediction is annotated with one of three options: *accept*, *reject*, or *reject all*. The *reject all* option is chosen when the annotator believes that the given events did not imply any user intents. We also annotate data to identify the presence of intents under specific observations, which is particularly helpful when the number of predictions is relatively low. Three separate annotators are enlisted to label each prediction of user intent. This process results in a dataset of $1,400$ entries, each containing event traces, predictions, and decisions on the acceptance of user intents from three annotators. Notably, our annotators achieve an impressive consistency rate of over 90.92%, underscoring the reliability of the annotations and the robustness of the dataset for further analysis.

Setting. We use the 1,400 task entries with human annotations and randomly split them into a training set (1,300) and a test set (100). We then train Llama-3-Instruct 8B due to its strong instruction-following ability on the training set to obtain our automatic evaluator. The detailed training hyper-parameters and the prompt template are listed Appendix A. We use the test split to evaluate

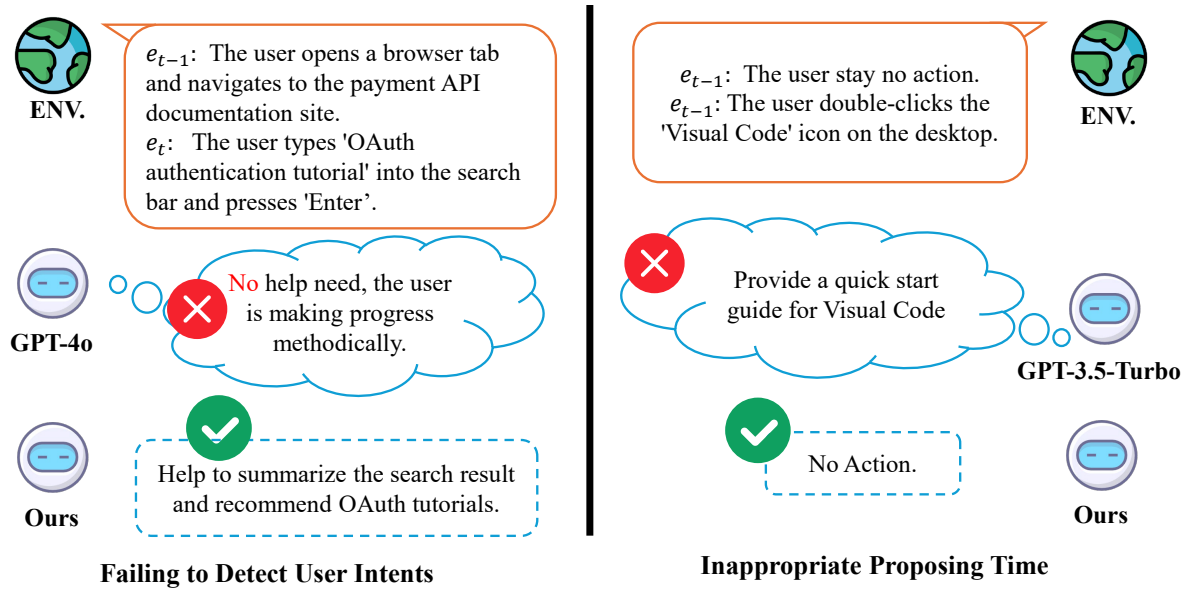


Figure 4: Two types of common failure: failing to detect user intents (left) and inappropriate proposing time (right).

our adapted model and all the baselines. During the evaluation, we prompt models to generate acceptance of each prediction and compare it with human-annotated results. This measures whether models could successfully align with human judgments regarding whether the proposed agent task is appropriate.

Results. We test several baseline models including GPT-4o, GPT-3.5-Turbo, Claude-3-Sonnet, LLaMA-3-70B, Qwen2 72B and Deepseek-v2. The results show that our model effectively aligns with human judgment, with an accuracy of 76% on our fine-tuned model. While other models cannot predict user behaviors effectively. Besides, among these models, LLaMA-3-70B achieves a pretty good result of 53%, defeating close source models like GPT-4o.

5.3 Case Study

In this section, we explore two prevalent types of failures encountered in predicting human intents: the inability to detect user intents and making predictions at inappropriate times. As illustrated in Figure 4 (left), a notable failure occurs when the GPT-4o model does not assist at crucial moments. For instance, when a user is engaged in integrating a payment API and requires a tutorial for guidance, the model remains silent. Instead, our model successfully detects human intents and offers assistance. The underlying intention—to minimize disruptions—ironically leads to missed opportunities for offering timely help. Conversely, the right side of Figure 4 showcases an instance of ill-timed prediction. Here, the GPT-3.5-Turbo suggests an action when there are no user intents shown in events. This scenario underscores the possible unintended events existing in human activities. The model should judge whether there are possible user intents smartly to avoid unnecessary actions. These instances highlight the intricate nature of human activities and the sophisticated reasoning required for models to accurately predict human intents. To navigate the delicate balance between being helpful and intrusive, models must develop a deeper understanding of user contexts and activities, ensuring their interventions are both timely and pertinent.

6 Conclusion

We introduce an innovative approach to human-agent interaction by leveraging proactive predictions of human intents based on sequences of environmental events. We introduce ProactiveBench, a dataset comprising 11,876 events, and establish an automatic benchmark to assess the performance of eight state-of-the-art language models and our model LLaMA-3-Proactive. Our findings highlight the persistent challenges in accurately and timely predicting human intents within this context. Future research should focus on enhancing the precision and timeliness of intent predictions to improve the efficacy of human-agent interactions.

Ethics Statement

In the construction of our dataset, we have adhered to strict ethical standards. The human annotators are students from the computer sciences area and obtain rewards. We use ChatGPT in this research. All the original data utilized for the compilation of our dataset are either sourced from open-access platforms or have been used with the appropriate permissions granted. We have conducted a thorough review to ensure that our dataset is devoid of any harmful content. Our commitment to ethical research practices extends to the respect for privacy, intellectual property rights, and the integrity of the data sources.

Limitations

Though our method demonstrates that language models can effectively and proactively predict human intents, the researched environment settings are still limited. Besides, our model still cannot perfectly predict the human intents as results show, presenting a relatively high ratio of false alarms. More future work should be done to build an agent that can more precisely predict human intents on time.

References

Anthropic. 2024. Introducing the claude-3 family. Accessed: 2024-06-16.

AutoGPT. 2023. Autogpt.

BabyAGI. 2023. Babyagi.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. *CoRR*, abs/2005.14165.

Weiwen Chen, Mohammad Shidujaman, Jiangbo Jin, and Salah Uddin Ahmed. 2020. A methodological approach to create interactive art in artificial intelligence. In *HCI International 2020 – Late Breaking Papers: Cognition, Learning and Games*, pages 13–31, Cham. Springer International Publishing.

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. *Preprint*, arXiv:2308.10848.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.

DeepSeek-AI. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *Preprint*, arXiv:2405.04434.

Roya Firoozi, Johnathan Tucker, Stephen Tian, Anirudha Majumdar, Jiankai Sun, Weiyu Liu, Yuke Zhu, Shuran Song, Ashish Kapoor, Karol Hausman, et al. 2023. Foundation models in robotics: Applications, challenges, and the future. *arXiv preprint arXiv:2312.07843*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. *Preprint*, arXiv:2211.10435.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of experts. *Preprint*, arXiv:2401.04088.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. *Preprint*, arXiv:2303.17760.

Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal llm agents: Insights and survey about the

capability, efficiency and security. *arXiv preprint arXiv:2401.05459*.

Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*.

Meta. 2024. Introducing meta llama 3: The most capable openly available llm to date.

OpenAI, :, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mo Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2023. Gpt-4 technical report. Technical report.

OpenAI. 2022. OpenAI: Introducing ChatGPT.

OpenAI. 2024. Hello gpt-4. Accessed: 2024-06-16.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *Preprint*, arXiv:2203.02155.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. *Preprint*, arXiv:2304.03442.

Chen Qian, Xin Cong, Wei Liu, Cheng Yang, Weize Chen, Yusheng Su, Yufan Dang, Jiahao Li, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023a. Communicative agents for software development. *Preprint*, arXiv:2307.07924.

Cheng Qian, Bingxiang He, Zhong Zhuang, Jia Deng, Yujia Qin, Xin Cong, Yankai Lin, Zhong Zhang, Zhiyuan Liu, and Maosong Sun. 2024. Tell me

more! towards implicit user intention understanding of language model driven agents. *arXiv preprint arXiv:2402.09205*.

Cheng Qian, Chenyan Xiong, Zhenghao Liu, and Zhiyuan Liu. 2023b. Toolink: Linking toolkit creation and using through chain-of-solving on open-source model. *arXiv preprint arXiv:2310.05155*.

Yujia Qin, Zihan Cai, Dian Jin, Lan Yan, Shihao Liang, Kunlun Zhu, Yankai Lin, Xu Han, Ning Ding, Huadong Wang, Ruobing Xie, Fanchao Qi, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023a. Webcpm: Interactive web search for chinese long-form question answering. In *Proceedings of ACL 2023*. Association for Computational Linguistics.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, et al. 2023b. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023c. Toolllm: Facilitating large language models to master 16000+ real-world apis. *Preprint*, arXiv:2307.16789.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Preprint*, arXiv:2302.04761.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models. *Preprint*, arXiv:2201.11903.

Christina Wiethof, Navid Tavanapour, and Eva Alice Christiane Bittner. 2021. Implementing an intelligent collaborative agent as teammate in collaborative writing: toward a synergy of humans and ai. In *Hawaii International Conference on System Sciences*.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation framework.

Zhuohao Wu, Danwen Ji, Kaiwen Yu, Xianxu Zeng, Dingming Wu, and Mohammad Shidujaman. 2021. Ai creativity and the human-ai co-creation model. In *Human-Computer Interaction. Theory, Methods and Tools*, pages 171–190, Cham. Springer International Publishing.

XAgent-Team. 2023. Xagent: An autonomous agent for complex task solving.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Preprint*, arXiv:2305.10601.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Yining Ye, Xin Cong, Yujia Qin, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2023a. Large language model as autonomous decision maker. *arXiv preprint arXiv:2308.12519*.

Yining Ye, Xin Cong, Shizuo Tian, Jiannan Cao, Hao Wang, Yujia Qin, Yaxi Lu, Heyang Yu, Huadong Wang, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2023b. Proagent: From robotic process automation to agentic process automation. *Preprint*, arXiv:2311.10751.

Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. 2022. Glm-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414*.

Appendix

A Evaluator Training Setting

We use Llama-3-Instruct 8B as the base model for our training. The total dataset size is approximately 1300. Specifically, we employ a total batch size of 16, learning rate $1e-5$, and Adam Optimizer with a 0.01 warm-up ratio. We train the evaluator for 3 epochs to prevent it from over-fitting. We use an A100 machine with 8 nodes to train for approximately 1.5 hours.

Template. We apply the following prompt template to train the evaluator:

Prompt Template

```
# Task
Evaluate the task proposed by the
    assistant.
## Rules
1. The accepted task should be
   relevant to the current
   situation.
2. The user prefers less
   intervention from the
   assistant, so only accepts the
    valuable task.\n\n## Output
   Format\n\"The comment of the
   proposed task based on the
   observation.
"true or false"
- `true` means the user `Accept`
   the task.
Example
- event: "The user types 'import
   numpy as np' in the first cell
    of the Jupyter notebook."
- proposed_task: "Offer a Python
   tutorial"
- **output**: "The user looks
   like he is capable of using
   Python, so the Python tutorial
    might not be necessary.
#### false"

## History Observations
{Action Track Observation History
   }

## Current Observation
{Current User Action Observation}

## Proposed Task
{Proposed Candidate Task}

## Instruction
Evaluate the task proposed by the
    assistant and simulate the
   user's response to it based on
    the given observations.
```

B Agent Model Training Setting

Similarly, we use Llama-3-Instruct 8B as the base model for agent model training. The total dataset size is approximately 9000. Specifically, we employ a total batch size of 32, learning rate $1e-5$, and Adam Optimizer with a 0.01 warm-up ratio. We train the evaluator for 3 epochs to prevent it from over-fitting. We use an A100 machine with 8 nodes to train for approximately 6.5 hours.

Template. We apply the following prompt template to train the agent model:

Prompt Template

```
You are a helpful assistant, and
   the user you are assisting is
   doing multiple actions, here
   are the user's atomic action
   tracks:
{Action Track Observation History
   }

And what's more, here is some
   background information:
{User Background Information}

Here is the proposition history
   of you and the user, True
   means you offered a proposal
   to the user, False otherwise,
   you should propose your
   assistance to the user at a
   pace of a half, which means in
    all the turns in the proposed
    history considered, you
   should only make a proposed at
    around half of the rounds.
proposed_history: {Proposal
   History}
```

Model Name	Category	C.R.%	F.R.%	A.R.%	P.R.%
GPT-4o	Coding	54.61	15.97	56.88	50.00
	Writing	26.67	30.91	47.93	16.67
	Smart Home	0.00	33.33	50.00	0.00
GPT-3.5-Turbo	Coding	30.26	14.44	64.36	26.97
	Writing	23.33	18.75	70.41	21.67
	Smart Home	12.50	38.10	41.54	8.33
Claude-3-Sonnet	Coding	26.97	6.45	51.47	25.00
	Writing	10.71	22.22	53.12	3.57
	Smart Home	0.00	0.00	0.00	0.00
DeepSeek-v2	Coding	59.87	16.56	54.78	51.32
	Writing	58.33	22.32	50.89	46.67
	Smart Home	0.00	61.54	25.64	0.00
Llama-3-8B	Coding	61.84	9.89	67.77	58.55
	Writing	13.33	17.86	45.24	13.33
	Smart Home	8.33	60.00	26.67	4.17
Llama-3-70B	Coding	78.29	11.89	57.56	69.08
	Writing	76.67	21.53	49.54	68.33
	Smart Home	0.00	14.29	50.00	0.00
Qwen2 72B	Coding	58.39	20.26	56.64	49.66
	Writing	69.84	39.74	35.78	50.79
	Smart Home	66.67	40.00	28.72	50.00
Mixtral 8x22B	Coding	63.16	21.51	53.11	51.97
	Writing	58.33	30.58	47.40	43.33
	Smart Home	16.67	56.52	28.99	4.17
ProactiveLlama	Coding	87.50	17.05	61.38	74.34
	Writing	91.67	26.40	58.16	83.33
	Smart Home	29.17	56.47	22.27	12.50

Table 4: Full results of agent evaluation.

```
In this scene, some tools are provided for you to better aid the user, and they are in the format of 'Appliance --> Functionality'
{Assistant Tools}

Based on the information above, you should deduce some information from it, which includes:
1. What is the user's purpose for doing the last action?
2. What is the user's personality and preference based on the action tracks?
3. Take all the events and propose history into consideration, if you can only offer your assistance when the user requires immediate help, do you think it is time to offer assistance to meet the user's requirements?
4. If you think it is a proper time to offer help, please provide EXACT 5 possible option(s) for the user.
Please take all the event information into consideration (including contents, time, and anything passed by the event), and answer the following questions by outputting a JSON format as below.
```json
{
  "purpose":[The purpose of the last action in string],
  "profile":[the profile and preference of the user deduced in string],
  "need_help":['yes' if you think you could offer assistance to the user at this time, or 'no' if you think it's not a proper time, followed by the reason in string]
```
```

```
  "raise_query": [a prompt that
     you generate to tell the
     user that you are going to
     help if you are to offer
     help, or an empty string if
     not]
  "candidate_task": [a list of
     choices written in natural
     language for humans, which
     are the possible ways you
     could do to help the user.
     Generate exactly 3
     candidates. If you find the
     given tools helpful and
     useful for the user, just
     don't hesitate to use them.
     If you are using the
     appliance provided, please
     describe your action using a
      tool to help in natural
     language as one choice. If
     it is not a proper time to
     offer assistance, return an
     empty list]
}
```

For example:
{An output Example}